

1.Small language model (SLM)

“ A small language model(slm) is an AI model that processes and generates answers,but uses fewer parameters (>1B) ”

Key characteristics :

Feature

Parameters	1M – 7B
Hardware needed	laptop /mobile/edge devices
Inference cost	low
Latency	fast
Customization	easy to fine- tune for a domain
Best for	narrow, specific tasks

Why SLMs matter :

- Cost-efficient — cheaper to train, fine-tune, and run.
- Privacy/on-device — can run locally (no data leaving the device), important for sensitive domains.
- Speed — low latency, good for real-time apps.
- Domain specialization — a small model fine-tuned on *your* niche data often beats a generic giant model on that narrow task.
- Sustainability — lower energy/compute footprint.

Common techniques used to build SLMs:

- Distillation — training a small "student" model to mimic a large "teacher" model.
- Pruning — removing less-important weights/neurons from a bigger model.
- Quantization — reducing precision (e.g., 32-bit → 8-bit/4-bit) to shrink size and speed up inference.

2. Domain : Education

Project idea :Language learning companions

SLM used : Gemma 2B or Phi-3-mini — both are small enough to fine-tune on consumer hardware (or even run via API), and both have decent multilingual conversational ability out of the box.

Problem it solves :

- Generating *new* conversations every time (not scripted/repetitive)
- Explaining grammar mistakes in real-time, in context
- Adjusting difficulty dynamically based on the learner's responses

Why only SLM :

- low latency
- on device / offline potential
- cost at scale
- privacy

Who would it used :

Students preparing for exams

Corporate training platforms

Language learning apps

3.Tokenizers :

What is tokenizers ?

A tokenizers is the process of converting a sentence into words .this acts as bridge between human language

Example : Text: "Healthcare" → Tokens: ["Health", "care"] → IDs: [154, 892]

4.Why tokenizers are needed ?

1. model processes on numbers not on text
2. vocabulary size limits to model to manage token set
- 3.for efficiency ,shorter sentence is equal to faster training

5. Types of tokenizers

Type	How It Works	Example	Pros	Cons
Word-level	Splits by spaces	"hello world" → ["hello", "world"]	Simple	Fails on unseen words
Character-level	Splits into chars	"hello" → ["h","e","l","l","o"]	Handles any word	Very long sequences
Subword-level	Splits into common subwords	"healthcare" → ["health", "care"]	Balances both	Complex implementation
Byte-level	Splits into bytes	Unicode → bytes	Universal	Very long

6. How tokenizers impacts on performance

Impact Factor	Explanation
Sequence Length	Poor tokenization → longer sequences → slower training arxiv
Vocabulary Coverage	Too small → many unknown words; too large → inefficient arxiv
Rare Word Handling	Subword tokenization handles rare words better
Model Accuracy	Better tokenization → better context understanding arxiv
Training Cost	Optimal tokenization reduces computational cost

